

Fullstack Development

Fullstack Landscape

| Diagram

Inspired from this [VDO](#).

Part 1: The battle of the architectures

MPA vs SPA vs HTMX

Multi-page application (MPA)

| The web's default architecture (1991–2000s)

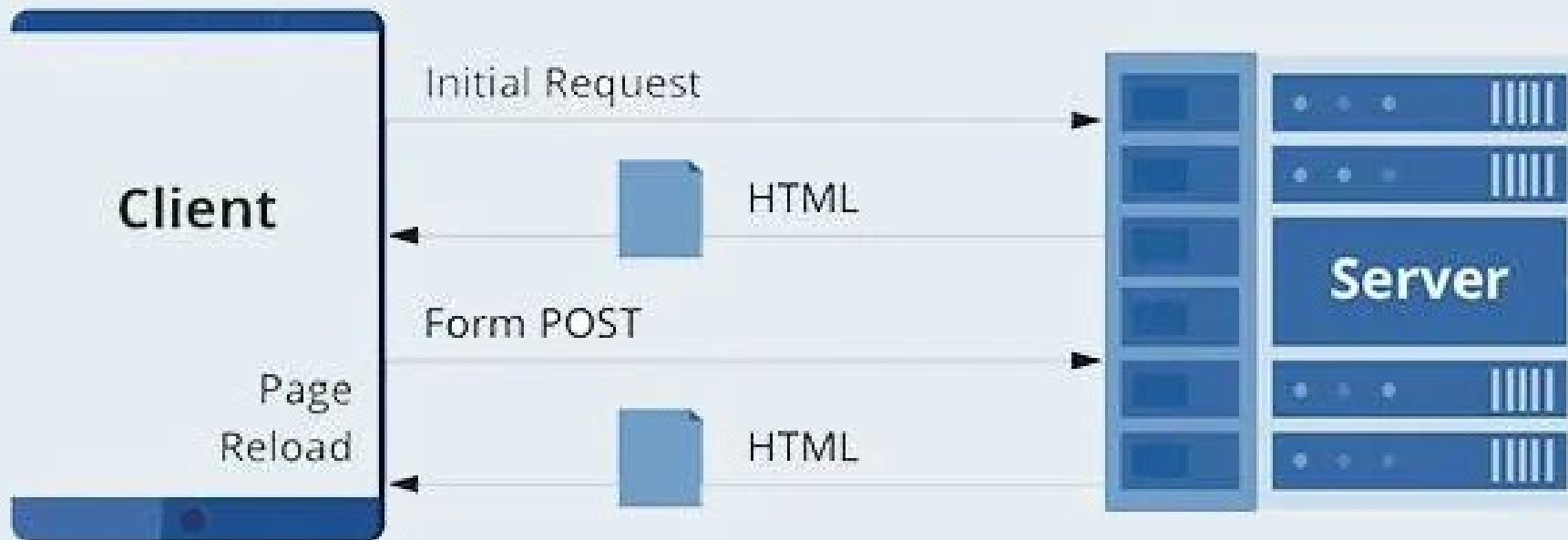
History

- 1990-91 : Tim Berners-Lee's WWW — static hypertext documents only
- 1993 : CGI — first standard way for a server to generate HTML dynamically
- 1995-99 : PHP, ASP, JSP — server-side scripting goes mainstream
- 2004-05 : Rails, Django — opinionated MVC frameworks standardize the pattern
- **Model:** every interaction = new HTTP request = full page reload

Multi-page application

- Loads a new page every time you perform an action.
- Traditional web applications.
- Use server-side technologies
 - PHP, Ruby on Rails, ASP.NET, Java, and Node JS.
- Can include JavaScript (`script`) for client-side interactivity

MPA Lifecycle



Todo app (MPA)

- Express JS + Pug (renderer)

Get started

- `git clone https://github.com/fullstack-69/ls-mpa.git`
- `cd ls-mpa`
- `pnpm install`
- `pnpm run build`
- `pnpm run start`

Endpoint

./src/index.ts

```
app.get("/", async (req, res) => {  
  const message = req.query?.message ?? "";  
  const todos = await getTodos();  
  // Output HTML  
  res.render("pages/index", {  
    todos,  
    message,  
    mode: "ADD",  
    curTodo: { id: "", todoText: "" },  
  });  
});
```

Renderer

```
./view/pages/index.pug
```

```
body
  main(class="container")
    a(href="/")
      h1 Todo (MPA)
    div(id="todoform")
      include ../components/inputform.pug
    div(id="todolist")
      include ../components/todolist.pug
```

The screenshot shows a web browser window with a single tab titled 'Todo'. The address bar shows 'localhost:3001'. The page displays a 'Todo (MPA)' application with a text input field, a 'Submit' button, and a list of todos. The first todo is '(1) 🍌 My First Todo' with delete and edit icons. The browser's developer tools are open to the 'Network' tab, showing a list of requests. A red box highlights the first three requests: 'localhost', 'pico.min.css', and 'pico.colors.min.css'. The 'Network' tab interface includes filters, a timeline, and a table of requests.

Name	Status	Type	Initiator	Size	T...
localhost	304	document	Other	943 B	5...
✓ pico.min.css	200	stylesheet	(index):0	(memory c...	0...
✓ pico.colors.min.css	200	stylesheet	(index):0	(memory c...	0...

3 requests | 943 B transferred | 158 kB resources | Finish: 537 ms | DOMContentLoaded: 538 ms | Load: 545 ms

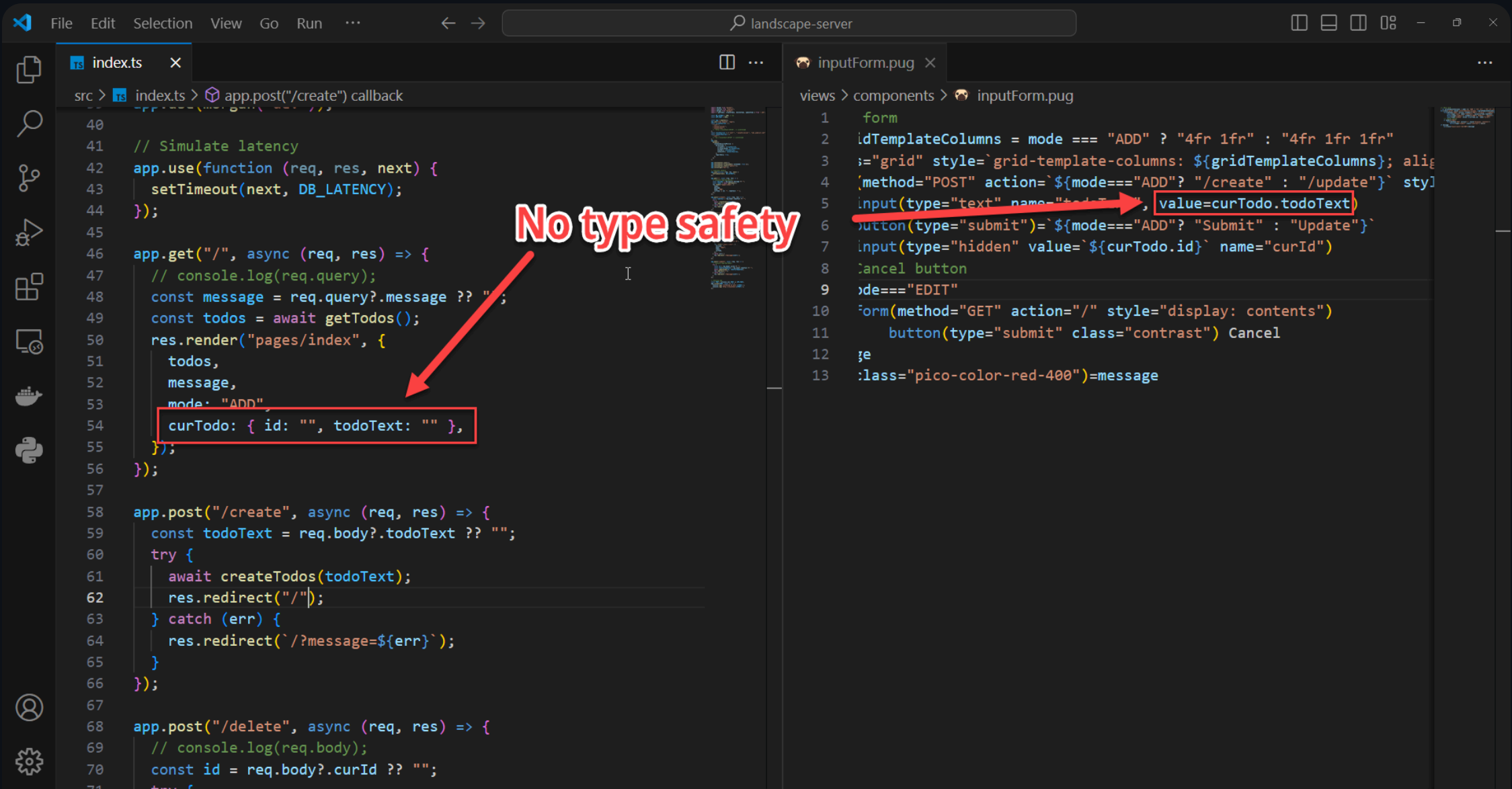
Use incognito mode to avoid loading chrome extensions.

261497: Fullstack Development

Note

- Every button is wrapped in a separate form
 - Need to trigger different endpoints.
- Need to use `input(type="hidden")` to encode additional information.

```
form(action="/delete" method="POST" style="display: contents")
  input(type="hidden" value={`${todo.id}`} name="curId")
  button(type="submit" class="contrast" style="margin-bottom: 0") 🗑️
form(action="/edit" method="POST" style="display: contents")
  input(type="hidden" value={`${todo.id}`} name="curId")
  button(type="submit" class="secondary" style="margin-bottom: 0") ✎
```



Single-page application (SPA)

■ The "thick client" era (2004–2010s)

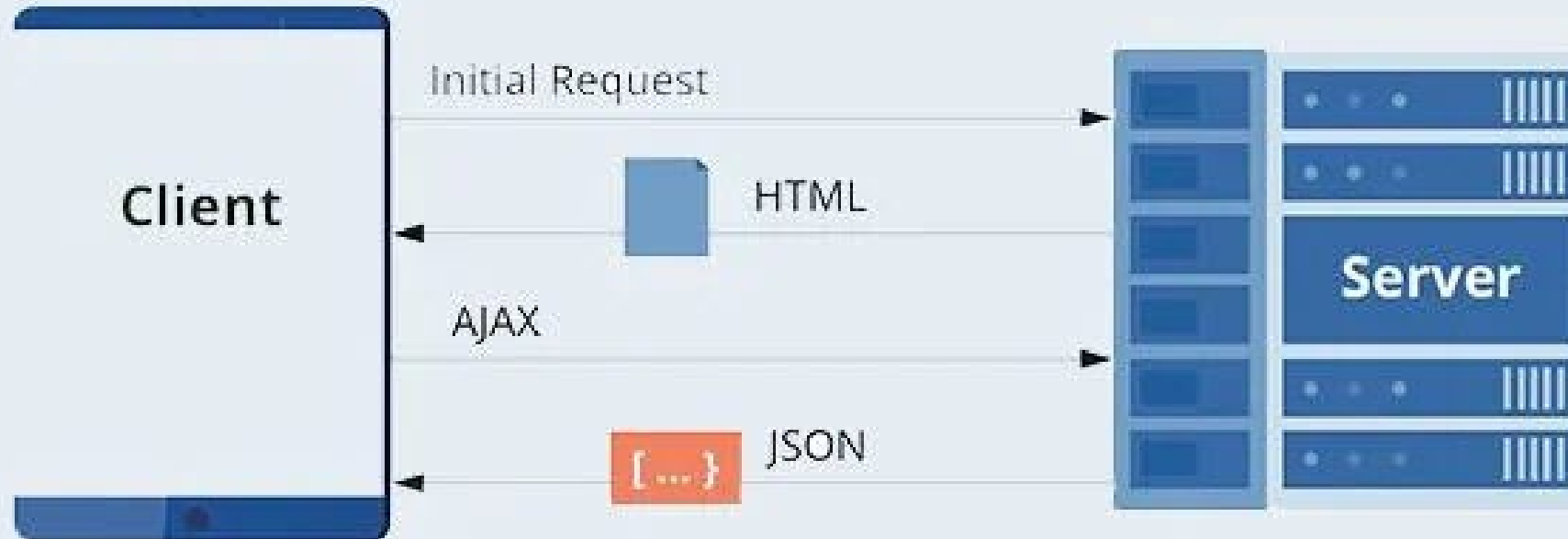
History

- 2004 : Gmail launches — async UI updates without page reloads
- Feb 2005 : Google Maps launches; Jesse James Garrett coins "Ajax"
- 2006 : jQuery — makes AJAX/DOM manipulation practical for everyone
- 2010-14 : Backbone/Knockout → AngularJS (2010) → React (2013) → Vue (2014)
- **Shift:** XML → JSON, server becomes an API, client owns rendering + state
- **Model:** one HTML shell, JavaScript renders everything client-side

Single-page application

- Single-page application
 - Loads a single HTML page and dynamically updates the content as the user interacts with the app.
- Use frontend and backend frameworks separately.

SPA Lifecycle



Todo app (SPA)

- Express JS + React

Get started

- `git clone https://github.com/fullstack-69/ls-spa.git`
- Backend
 - `cd ls-spa/backend`
 - `pnpm install`
 - `pnpm run build`
 - `pnpm run start`

Get started (cont.)

- Frontend

- `cd ls-spa/frontend`

- `pnpm install`

- `pnpm run build`

- `pnpm run preview`

Vite + React + TS

localhost:5001

Todo (SPA)

Submit

(1) 🍌 My First Todo ✎ 🗑

Network

Filter

All Fetch/XHR Doc CSS JS Font Img Media Manifest WS Wasm Other

Blocked requests 3rd-party requests

100 ms 200 ms 300 ms 400 ms 500 ms 600 ms

Name

- localhost
- index-uMjx6kOj.js
- index-z6Xg7xcP.css
- todo
- vite.svg

Headers Preview Response Initiator >>

```
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8" />
5     <link rel="icon" type="image/svg+xml"
6     <meta name="viewport" content="width=
7     <title>Vite + React + TS</title>
8     <script type="module" crossorigin src
9     <link rel="stylesheet" crossorigin hr
10  </head>
11  <body>
12    <main id="root"></main>
13  </body>
14 </html>
15
```

5 requests | 1.5 kB transferred | 343 kB resources | Finish: 5

Edit Selection View Go ... landscape-spa

App.tsx X TS types.ts X

frontend > src > App.tsx > App

```
1 import { useEffect, useState } from "react";
2 import axios from "axios";
3 import { type TodoType } from "../utils/types";
4 import { FormInput } from "../components/FormInput";
5 import { TodoList } from "../components/TodoList";
6 import { Spinner } from "../components/Spinner";
7
8 function App() {
9   // Fetching data
10  const [todos, setTodos] = useState<TodoType[]>([]);
11  async function fetchData() {
12    const res = await axios.get<TodoType[]>("api/todo");
13    setTodos(res.data);
14  }
15  useEffect(() => {
16    fetchData();
17  }, []);
18 }
```

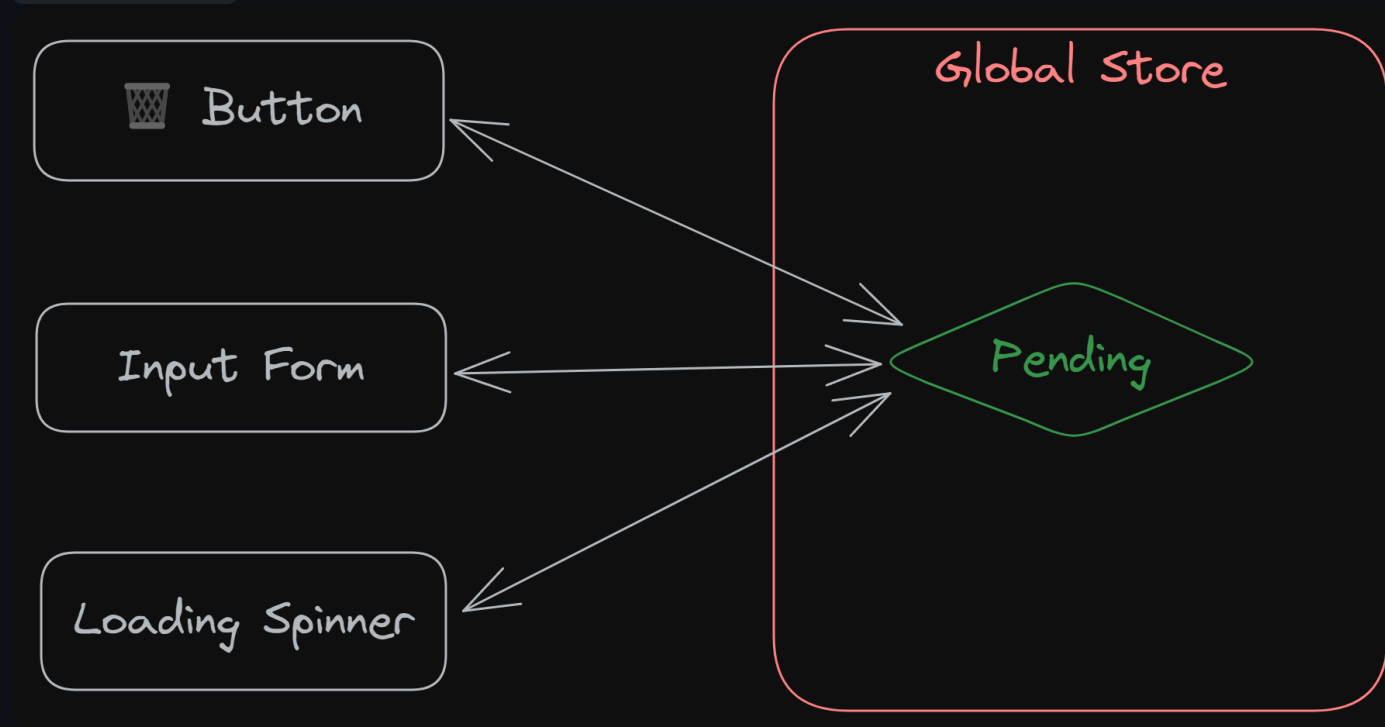
frontend > src > utils > TS types.ts > ...

```
1 export interface TodoType {
2   id: string;
3   todoText: string;
4 }
5
```

Not safe

Global store

- zustand



HTMX

■ The hypermedia revival (2013–present)

What exactly is HTMX?

Small JS library that can swap parts of UI with *HTML response* from a server.

#1 2024 JavaScript Rising Stars

History

- 2013 : Carson Gross builds Intercooler.js to fix slow client-side table sorting
 - Users tell him: "this is RESTful" → sends him back to Roy Fielding's dissertation
 - Fielding's 2008 essay: most "REST APIs" ignore HATEOAS (hypermedia constraint)
- Nov 24, 2020 : rewritten and relaunched as htmx — drops jQuery dependency

History

- 2022–23 : viral adoption, framed as reaction to SPA/build-tooling fatigue
July 2023: Hypermedia Systems book — names the category "Hypermedia-Driven Application"
- **Model:** server renders HTML fragments, JS library swaps them into the DOM

Get started

- `git clone https://github.com/fullstack-69/ls-htmx.git`
- `cd ls-htmx`
- `pnpm install`
- `pnpm run build`
- `pnpm run start`

Todo app

- The stack is very similar to MPA (`express` + `pug`).
- SSR enabled (*duh!*)
- No reloading
- Spinner enabled
- Form input disabled during submission.

In addition

- Hypermedia as the Engine of Application State (HATEOAS)
- HTMX + Alpine.js

A REST Paradox

	SPA backend	HTMX backend
HTTP verbs	GET / PUT / PATCH / DELETE	POST /edit, POST /delete, GET /cancel
Response	JSON — client needs out-of-band API knowledge	HTML — response <i>is</i> the next set of possible actions

- SPA satisfies "REST" by convention but violates Fielding's **HATEOAS** constraint
- HTMX looks verb-sloppy but is fully **hypermedia-driven** ✓

"Most REST APIs ignore HATEOAS" — Roy Fielding, 2008

App Comparison

UX

Item	MPA	SPA	HTMX
No page-refresh	✗	✓	✓
Spinner	✗	✓	✓
Element disabling	✗	✓	✓

App comparison (technical)

Item	MPA	SPA	HTMX
Amount of JS loaded	✓	✗	✓
HTML content (SEO)	✓	✗	✓
State in URL	✓	✗	✓

DX

Item	MPA	SPA	HTMX
Number of frameworks	1 	2 	1 
Complexity	Less 	More 	Less 
Lines of code	Less 	More 	Less 
Type Safety	Less 	More 	Less 
Hot reloading	Partial 	Full 	Partial 

Hot-Reload: A Clarification

- Both MPA and HTMX use identical `nodemon` configs — restarts the Node process on file change
- The SPA's full HMR comes from **Vite**, not from being an SPA

"Server-rendered apps restart-and-manual-refresh; only the Vite-powered frontend gets instant HMR"

Code Count

Architecture	Lines
MPA	~180
HTMX	~260
SPA	~420

- HTMX has **more code than MPA** — fragment templates and indicator wiring add lines that plain MPA never needs
- The bigger win shows up as apps grow: MPA/HTMX scale linearly with routes; SPA carries fixed framework overhead from the start

Summary

Architecture	Usage
MPA	Backend-critical (financial, ERP, ဂဏ္ဍီဃု)
SPA	Dashboards, editor
HTMX	Apps that finish on time

Appendix

Code walkthroughs: MPA → SPA → HTMX.

This part is AI-generated and is not used for tutorials.

Shared Foundation

All three apps simulate the same 1-second DB latency — making UX differences undeniable in a live demo:

```
// src/index.ts – identical in all three projects
const DB_LATENCY = 1000;
app.use(function (req, res, next) {
  setTimeout(() => next(), DB_LATENCY);
});
```

Every button press visibly stalls for a second. This is the forcing function that makes the UX comparison real, not hypothetical.

MPA: Project Structure

```
ls-mpa/  
├── src/  
│   ├── db.ts           # In-memory CRUD store  
│   └── index.ts        # Express server – 5 endpoints  
└── views/  
    ├── pages/  
    │   └── index.pug    # Full page layout  
    └── components/  
        ├── inputForm.pug # Add/Edit form (mode-aware)  
        └── todoList.pug  # List with hidden-input buttons
```

One server. One template engine. No browser JavaScript.

MPA: Route Pattern

Every action → POST to a dedicated endpoint → `res.redirect("/")`:

```
// src/index.ts
app.post("/create", async (req, res) => {
  await createTodos(req.body?.todoText ?? "");
  res.redirect("/"); // full page reload
});

app.post("/delete", async (req, res) => {
  await deleteTodo(req.body?.curId ?? "");
  res.redirect("/"); // full page reload
});
```

The browser re-fetches everything — HTML, CSS, the entire todo list — on every mutation.

MPA: The Hidden-Input Pattern

HTML forms can only send their own fields. To attach a todo's `id` to a delete/edit button, each button gets its own `<form>` with a hidden input:

```
//- views/components/todoList.pug
form(action="/delete" method="POST" style="display: contents")
  input(type="hidden" value=`${todo.id}` name="curId")
  button(type="submit") 🗑️

form(action="/edit" method="POST" style="display: contents")
  input(type="hidden" value=`${todo.id}` name="curId")
  button(type="submit") ✎
```

No JavaScript. Each button fires a separate form submit to a different endpoint.

MPA: The Edit Flow

Clicking  re-renders the same page in EDIT mode with the selected todo pre-filled:

```
// src/index.ts
app.post("/edit", async (req, res) => {
  const curTodo = todos.find((el) => el.id === id);
  res.render("pages/index", {
    mode: "EDIT", // ← toggles form UI
    curTodo, // ← pre-fills the input
    todos,
  });
});
```

The template reads `mode` to choose between "Submit"/"Update" labels and whether to show a Cancel button.

MPA: Full Page Layout

```
// - views/pages/index.pug
body
  main(class="container")
    a(href="/")
      h1 Todo (MPA)
    div(id="todoform")
      include ../components/inputform.pug
    div(id="todolist")
      include ../components/todolist.pug
    // - LivereLoad
    // - script(type='text/javascript' src="script.js")
```

The commented-out livereload script is a sign that someone tried to add dev-time browser refresh and gave up — a characteristic limitation of the plain server-rendering workflow.

SPA: Project Structure

```
ls-spa/  
├── backend/src/  
│   ├── db.ts           # Same in-memory store  
│   └── index.ts        # JSON API: GET/PUT/PATCH/DELETE /todo  
└── frontend/src/  
    ├── main.tsx        # React entry point  
    ├── App.tsx         # Root: fetches todos, composes UI  
    ├── components/  
    │   ├── FromInput.tsx # Form with manual pending state  
    │   ├── TodoList.tsx  # List with Axios delete/edit  
    │   └── Spinner.tsx   # Reads pending from global store  
    └── utils/  
        ├── store.ts     # Zustand: mode, pending, inputText, curId  
        └── types.ts      # Shared TodoType
```

Two separate codebases. Two `pnpm install` commands. Two servers in dev.

SPA: The Empty Shell

`frontend/index.html` — the entire initial HTML sent to the browser:

```
<body>
  <div id="root"></div>
  <script type="module" src="/src/main.tsx"></script>
</body>
```

This is what a search crawler — and a user on a slow connection — sees before JavaScript executes. No content. No SEO.

SPA: JSON REST API

backend/src/index.ts — proper HTTP verbs, JSON responses:

```
app.get("/todo", async (req, res) => res.json(await getTodos()));
app.put("/todo", async (req, res) => {
  /* create */ res.json({ msg: "..."});
});
app.delete("/todo", async (req, res) => {
  /* delete */ res.json({ msg: "..."});
});
app.patch("/todo", async (req, res) => {
  /* update */ res.json({ msg: "..."});
});
```

The backend is fully decoupled from the UI. It knows nothing about how data will be displayed.

SPA: Global State (Zustand)

frontend/src/utils/store.ts — one store for all UI state:

```
interface TodoStoreState {  
  mode: "ADD" | "EDIT";  
  curTodo: TodoType;  
  pending: boolean; // drives input/button disabling  
  inputText: string;  
  curId: string;  
  // ... setters  
}
```

`pending` is the key field: when `true`, inputs and buttons are disabled. This is the manual equivalent of HTMX's declarative `hx-disabled-elt`.

SPA: Manual Loading State

frontend/src/components/FromInput.tsx :

```
function handleSubmit() {  
  setPending(true); // ← manually disable UI  
  axios  
    .request({ url: "/api/todo", method: "put", data: { todoText: inputText } })  
    .then(fetchData) // ← re-fetch full todo list  
    .then(() => {  
      setInputText("");  
      setMessage("");  
    })  
    .catch((err) => alert(err))  
    .finally(() => setPending(false)); // ← manually re-enable UI  
}
```

Every state transition is hand-coded. Compare to HTMX's `hx-indicator` + `hx-disabled-elt` — same UX, zero JS.

HTMX: Project Structure

```
ls-htmx/  
├── src/  
│   ├── db.ts           # Same in-memory store  
│   └── index.ts        # Express – routes return HTML fragments  
└── views/  
    ├── pages/  
    │   └── index.pug    # Full page (initial load only)  
    └── components/  
        ├── wrapperFormList.pug # Swap target: form + list combined  
        ├── inputForm.pug      # Form with hx-* attributes  
        └── todoList.pug       # List items with hx-* attributes
```

Same stack as MPA. The key difference: routes return HTML fragments instead of redirecting.

HTMX: One Script Tag

`views/pages/index.pug` — the entire client-side JS dependency:

```
head
  script(
    src="https://cdn.jsdelivr.net/npm/htmx.org@2.0.10/dist/htmx.min.js"
    crossorigin="anonymous"
  )
```

No `npm install` for the frontend. No bundler. No build step. ~14 KB total.

HTMX: Swap Target Architecture

`views/pages/index.pug` — rendered only once, on first load:

```
body(class="container")
  main(class="container")
    a(href="/")
    h1 Todo (HTMX)
    div(id="wrapperFormList")
      include ../components/wrapperFormList.pug
    div(class="spinner-wrapper")
      div(id="spinner" class="lds-dual-ring htmx-indicator")
```

After the first load, only `#wrapperFormList` is ever replaced. The heading and spinner are permanent fixtures — never re-fetched.

HTMX: The Fragment Wrapper

`views/components/wrapperFormList.pug` — exists purely to give HTMX a swap target:

```
div(id="todoform")
  include ../components/inputForm.pug
div(id="todolist")
  include ../components/todoList.pug
```

This extra template is the price of no-refresh updates. The MPA never needs it. The server now must know how to render both a full page (initial load) and this reusable fragment (subsequent updates).

HTMX: Declarative Loading State

views/components/inputForm.pug :

```
form(  
  hx-post="/create"  
  hx-target="#wrapperFormList"  
  hx-indicator="#spinner"  
  hx-disabled-elt="find input[type='text'], find button"  
)  
  input(type="text" name="todoText")  
  button(type="submit") Submit
```

- `hx-indicator="#spinner"` — shows the spinner while the request is in-flight
- `hx-disabled-elt` — disables inputs and buttons automatically
- **Zero JavaScript written.** The SPA's `setPending(true/false)` logic doesn't exist here.

HTMX: Server Returns HTML Fragments

`src/index.ts` — instead of redirecting, every route renders the fragment:

```
app.post("/create", async (req, res) => {
  await createTodos(req.body?.todoText ?? "");
  const todos = await getTodos();
  res.render("components/wrapperFormList", {
    // ← HTML fragment
    todos,
    mode: "ADD",
    message: "",
    curTodo: blankTodo,
  });
});
```

HTMX: Server Returns HTML Fragments

HTMX receives the HTML and swaps it into `#wrapperFormList`. No JSON. No client-side state management. The server is the single source of truth for both data and UI state.

HTMX: The Cancel Endpoint

Cancel is just fetching the "clean" ADD-mode fragment from the server — no client-side state reset needed:

```
// src/index.ts
app.get("/cancel", async (req, res) => {
  const todos = await getTodos();
  res.render("components/wrapperFormList", {
    todos,
    mode: "ADD",
    message: "",
    curTodo: blankTodo,
  });
});
```

HTMX: The Cancel Endpoint

```
//- inputForm.pug  
button(class="contrast" hx-get="/cancel" hx-target="#wrapperFormList") Cancel
```

The server determines what "cancelled" looks like. The client just requests it.